



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV SUD

T.Y. B. TECH

Year: 2025 – 26

Semester: 6

Name of the Student: Mrinmayee Pramod
Kulkarni

PRN: 124B2B025

Branch: Computer Engineering

Division: C

Date: 16/2/2026

Assignment 4

Assignment Title:

Create any interactive data visualization dashboard for any real-life application like weather dashboard.
Visualize data using charts and graphs (use libraries like Chart.js or D3.js).

Objectives

- To understand the structure of a dynamic data visualization dashboard
- To learn how HTML is used to create layout and dashboard components
- To apply CSS and Bootstrap for styling and responsive UI
- To use JavaScript for fetching and visualizing real-time data

- To implement charts and graphs using Chart.js
- To design an interactive and user-friendly weather dashboard

Outcomes

- Design a functional weather dashboard interface
- Display weather data dynamically using charts and cards
- Apply responsive design using Bootstrap CDN
- Implement interactivity using JavaScript and APIs
- Improve understanding of front-end development and data visualization

Theory

An Agriculture Data Visualization Dashboard is a web-based application used to display and analyze farming-related data such as:

- Crop production
- Seasonal data
- Rainfall levels
- Crop yield

This helps farmers, researchers, and policymakers make data-driven decisions.

In this project, the dataset contains:

Field	Value
-------	-------

Crop	Wheat
------	-------

Year	2021
------	------

Season	Rabi
--------	------

Rainfall	400 mm
----------	--------

Yield	2600 kg/ha
-------	------------

Role of HTML

HTML provides the structure of the agriculture dashboard.

It is used to create:

- Header (Dashboard title)
- Crop information cards
- Data display sections
- Chart containers
- Footer

HTML organizes agricultural data in a structured format for users.

Role of CSS

CSS improves the visual presentation of the agriculture dashboard:

- Layout design (grid system)
- Color themes (green agriculture theme)

- Typography
- Card design
- Hover effects

CSS makes the dashboard attractive and readable.

Role of JavaScript

JavaScript makes the dashboard dynamic and interactive:

- Fetch crop data from database (MongoDB ObjectId given)
- Display rainfall and yield values dynamically
- Update charts based on year or season
- Handle filtering (crop-wise, year-wise)
- Perform basic analytics (average yield, rainfall trend)

Use of Chart.js

Chart.js is used to convert agriculture data into visual charts:

- Bar Chart → Yield per crop
- Line Chart → Rainfall over time
- Pie Chart → Seasonal crop distribution

This helps users quickly understand trends and patterns.

Use of Bootstrap CDN

Bootstrap helps make the dashboard responsive and mobile-friendly:

- Grid system for layout
- Responsive navbar
- Cards for crop data
- Buttons and dropdown filters
- Mobile-friendly design

Program code with output screenshot

Code:

Index.html

```
<!DOCTYPE html>

<html>

<head>

<title>Crop Yield</title>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<style>

body{background:#111;color:#fff;text-align:center}

canvas{max-width:500px;margin:30px auto;display:block}

</style>

</head>

<body>
```

```
<h2>Crop Yield Chart</h2>
```

```
<canvas id="chart1"></canvas>
```

```
<canvas id="chart2"></canvas>
```

```
<script>
```

```
fetch("/data")
```

```
.then(r=>r.json())
```

```
.then(d=>{
```

```
  console.log("DB DATA:",d);
```

```
  new Chart(chart1,{
```

```
    type:"bar",
```

```
    data:{
```

```
      labels:d.map(x=>x.year),
```

```
      datasets:[{
```

```
        label:"Yield",
```

```
        data:d.map(x=>x.yield)
```

```
      ]
```

```
    }
```

```
  });
```

```
new Chart(chart2,{

  type:"line",

  data:{

    labels:d.map(x=>x.year),

    datasets:[{

      label:"Rainfall",

      data:d.map(x=>x.rainfall)

    }]

  }

});

})

.catch(err=>console.error("ERROR:",err));

</script>

</body>

</html>
```

Server.js:

```
const express=require("express");
```

```
const mongoose=require("mongoose");

const app=express();

mongoose.connect("mongodb://localhost:27017/agriDB");

app.use(express.static("public"));

app.get("/data",async(req,res)=>{

  const data=await mongoose.connection.db

    .collection("crop_yield")

    .find({})

    .toArray();

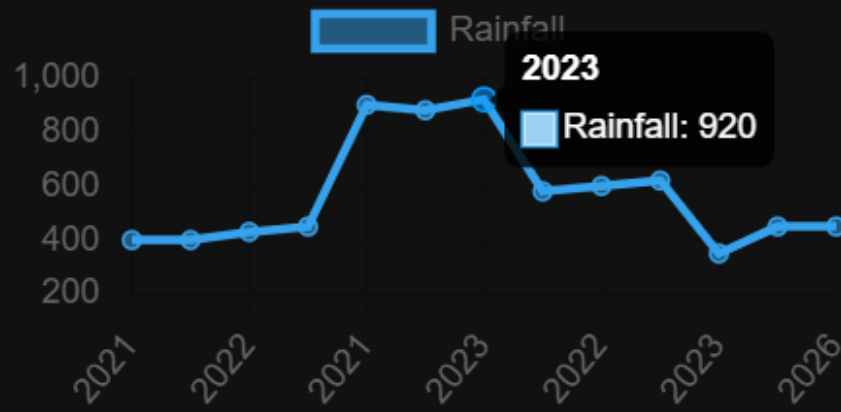
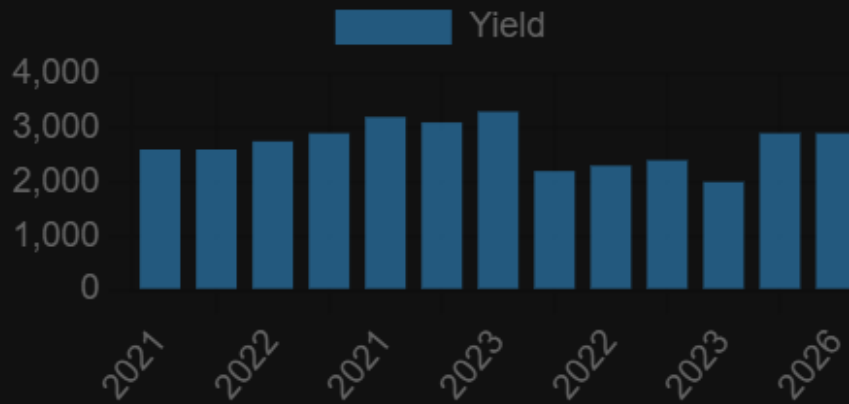
  res.json(data);

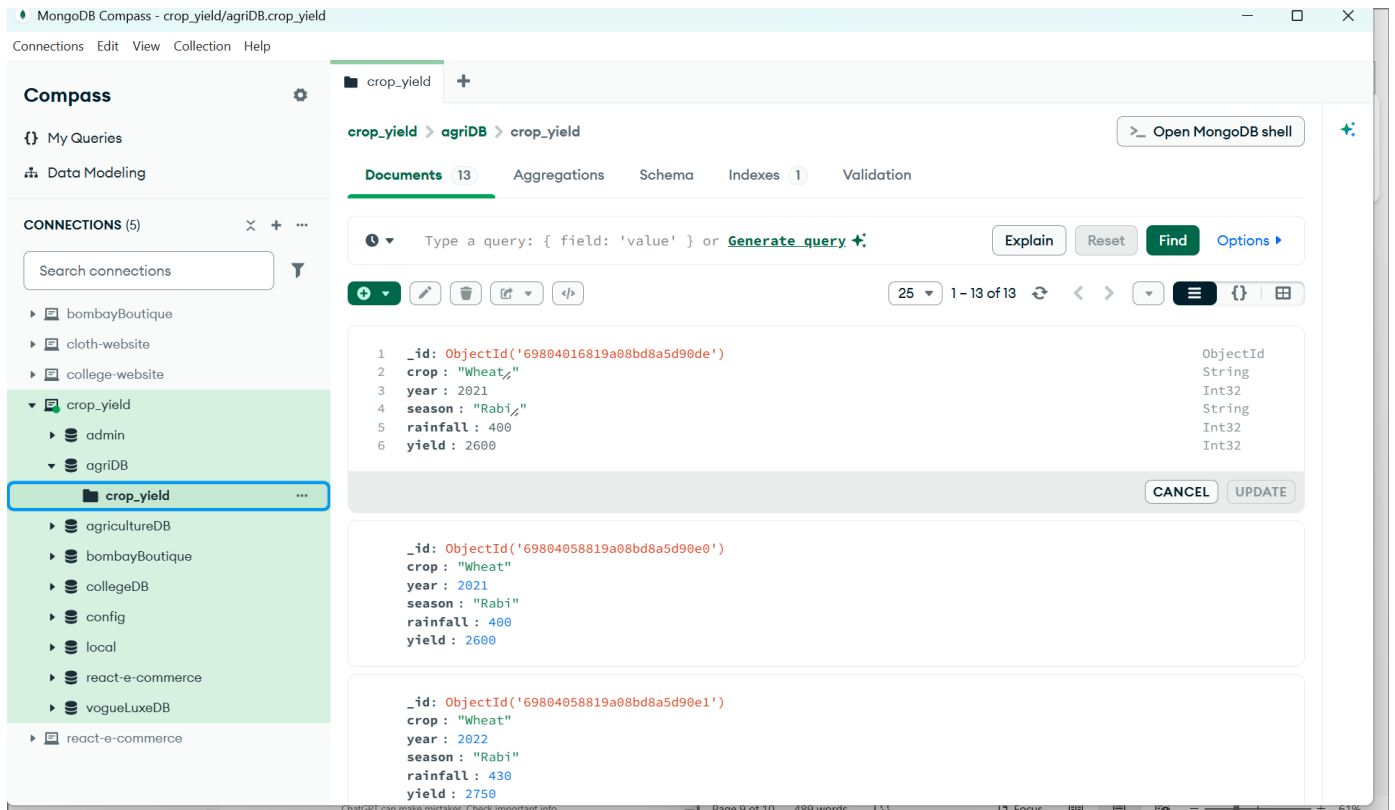
});

app.listen(3000,()=>console.log("Server running on 3000"));
```

Output:

Crop Yield Chart





Conclusion

The Agriculture Dashboard demonstrates how modern web technologies like HTML, CSS, Bootstrap, JavaScript, and Chart.js can be used to analyze real agricultural datasets.